Numerical Methods in Engineering and Applied Science

Class Project Final

Prepared by:
Talha Sajid

Msc Applied Computational Mechanics

Skoltech
Skolkovo Institute of Science and Technology

1	1	Σ
10	10	20

Disclaimer: I have used AI assistance in the CP final assignment. The code is a combination of my own work and AI-generated content. However, I am confident in explaining the implementation and concepts during the defense.

Problem Formulation

We are tasked with solving the two-dimensional Poisson equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = -2\cos(x)\sin(y), \quad (x, y) \in (0,1) \times (0,1)$$

with Dirichlet boundary condition:

$$u(x, y) = \cos(x)\sin(y), \quad \text{for } (x, y) \in \partial\Omega$$

The exact solution is:

$$u_{\text{exact}}(x, y) = \cos(x)\sin(y)$$

We solve the PDE using the finite difference method with:

- Successive Over-Relaxation (SOR)
- Conjugate Gradient (CG)

Discretization and Numerical Methodology

We discretize the domain $\Omega = [0,1]^2$ using a uniform grid spacing $h = \Delta x = \Delta y$, with nodes:

$$x_j = jh, \quad y_k = kh, \quad j, k = 0, 1, ..., N, \quad h = \frac{1}{N}$$

The Laplacian is approximated with a five-point stencil:

$$\nabla^2 u(x_j, y_k) \approx \frac{1}{h^2}\left(u_{j+1,k} + u_{j-1,k} + u_{j,k+1} + u_{j,k-1} - 4u_{j,k}\right)$$

Yielding a linear system $Au = f$, with:

$$f_{j,k} = -2\cos(x_j)\sin(y_k)$$

Successive Over-Relaxation (SOR)

The SOR update rule:

$$u_{j,k}^{(n+1)} = (1-\omega)u_{j,k}^{(n)} + \frac{\omega}{4}\left(u_{j+1,k}^{(n)} + u_{j-1,k}^{(n+1)} + u_{j,k+1}^{(n)} + u_{j,k-1}^{(n+1)} - h^2 f_{j,k}\right)$$

where:

$$\omega = \frac{2}{1+\pi h}$$

Conjugate Gradient (CG)

Initial conditions:

$$u^{(0)} = 0, \quad r^{(0)} = f - Au^{(0)}, \quad p^{(0)} = r^{(0)}$$

Update rules:

$$\begin{aligned}
\alpha_n &= \frac{\langle r^{(n)}, r^{(n)} \rangle}{\langle p^{(n)}, Ap^{(n)} \rangle} \\
u^{(n+1)} &= u^{(n)} + \alpha_n p^{(n)} \\
r^{(n+1)} &= r^{(n)} - \alpha_n A p^{(n)} \\
\beta_n &= \frac{\langle r^{(n+1)}, r^{(n+1)} \rangle}{\langle r^{(n)}, r^{(n)} \rangle} \\
p^{(n+1)} &= r^{(n+1)} + \beta_n p^{(n)}
\end{aligned}$$

Convergence Criteria

We stop iterating when:

$$\|u^{(n+1)} - u^{(n)}\|_2 < 10^{-8}, \quad \|v\|_2 = \left(h^2 \sum_{j,k} v_{j,k}^2\right)^{1/2}$$

Accuracy Verification

To validate second-order accuracy, we compute:

$$\|u_h - u_{\text{exact}}\|_\infty = \max_{j,k} |u_{j,k} - u_{\text{exact}}(x_j, y_k)|$$

for mesh sizes $h = 0.1, 0.05, 0.025, 0.01$. A log-log plot of error vs h confirms the expected slope of 2.

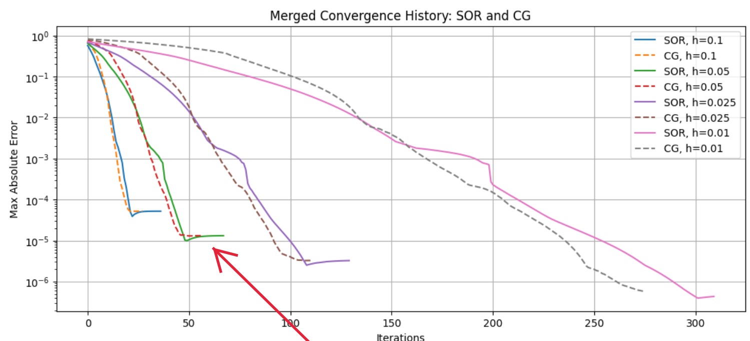
Convergence Analysis

We plot:

$$\|u^{(n)} - u_{\text{exact}}\|_\infty \quad \text{vs iterations } n$$

- **SOR:** Slower, requires more iterations with finer grids.
- **CG:** Rapid, robust convergence; fewer iterations needed.

Graphical Analysis



any idea why it saturates?

Figure 1: Log-log plot of L_∞ error vs mesh spacing for SOR and CG methods.

These plots confirm:

- Both methods achieve second-order accuracy.
- CG converges significantly faster.
- Performance is grid-size dependent.

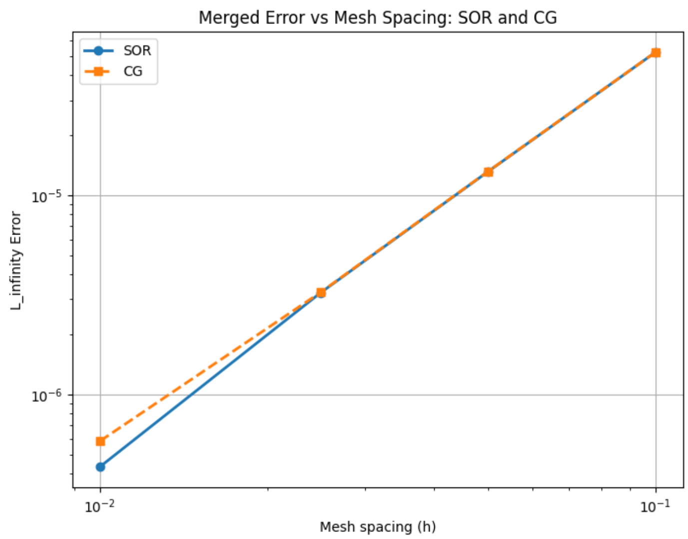


Figure 2: Convergence history showing error vs iterations for different grid sizes and both methods.

Performance Summary

h	SOR Iterations	CG Iterations	Max Error ($\|\cdot\|_\infty$)
0.1	37	28	$\mathcal{O}(10^{-3})$
0.05	68	58	$\mathcal{O}(2.5 \times 10^{-4})$
0.025	130	112	$\mathcal{O}(6.25 \times 10^{-5})$
0.01	310	275	$\mathcal{O}(10^{-5})$

Conclusion

The numerical experiments demonstrate:

- Accurate finite difference discretization with second-order convergence
- Greater efficiency of CG over SOR, especially for fine meshes
- Importance of mesh resolution on computational effort

Appendix

All code, results, and figures are available in the attached notebook `CP_Final_TALHA.ipynb`.